

Develop code that declares both static and non-static methods, and - if appropriate - use method names that adhere to the JavaBeans naming standards. Also develop code that declares and uses a variable-length argument list.

[Prev](#)

Chapter 1. Declarations, Initialization and Scoping

[Next](#)

Develop code that declares both static and non-static methods, and - if appropriate - use method names that adhere to the JavaBeans naming standards. Also develop code that declares and uses a variable-length argument list.

To set or get an instance variable, use methods with the names `setVariable(...)` and `getVariable()` for variables named `variable`:

```
private Object variable;

public void setVariable(Object var) {...}
public Object getVariable() {...}
```

For boolean instance variables you may provide a getter method named `is...` or `has...` that returns a boolean, that can be used conveniently in boolean expressions:

```
private boolean enabled;

public void setEnabled(boolean aBoolean) {...}
public boolean isEnabled() {...}
```

Varargs

In past releases, a method that took an arbitrary number of values required you to create an array and put the values into the array prior to invoking the method.

It is still true that multiple arguments must be passed in an array, but the varargs feature automates and hides the process. Furthermore, it is upward compatible with preexisting APIs. So, for example, a method now has this declaration:

```
method (type a, type b, type ... arguments);
```

The three periods after the final parameter's type indicate that the final argument may be passed as an array or as a sequence of arguments. Varargs can be used only in the final argument position.

See the following program for example:

```
public class VarArgs {

    public static void main(String... args) {
        printUs("ONE", "TWO", "THREE");
        printUs("FOUR", "FIVE");
        printUs(new String[]{"SIX", "SEVEN"});
        printUs(); // empty array is allowed too
    }

    private static void printUs(String... args) {
        System.out.println("Var args method");
        for (String s : args) {
            System.out.println(s);
        }
    }
}
```

```

        private static void printUs(String arg1, String arg2) {
            System.out.println("Specific two argument method");
            System.out.println(arg1);
            System.out.println(arg2);
        }
    }
}

```

It produces the following output:

```

Var args method
ONE
TWO
THREE
Specific two argument method
FOUR
FIVE
Var args method
SIX
SEVEN
Var args method

```

Purpose: add methods that can be called with variable-length argument list.

Heavily employed in formatting text output, aiding internationalization.

Syntax and semantics:

- the last formal parameter in a method declaration can be declared as:

```
ReferenceType ... FormalParameterName
```

- the last formal parameter in the method is then interpreted as having the type:

```
ReferenceType[]
```

```

// Method declaration
public static void publish(String str, Object ... data) // Object[]

```

```

// Method calls
publish("one");                // ("one", new Object[] {})
publish("one", "two");          // ("one", new Object[] {"two"})
publish("one", "two", 3);        // ("one", new Object[] {"two", new Integer(3)})

```

Some more varargs examples:

```

import static java.lang.System.out;

public class VarargsDemo {

    public static void main(String ... args) {
        int day = 1;
        String month = "February";
        int year = 2005;
    }
}

```

```

        flexiPrint(); // new Object[] {}
        flexiPrint(day); // new Object[] {new Integer(day)}
        flexiPrint(day, month); // new Object[] {new Integer(day), month}
        flexiPrint(day, month, year); // new Object[] {new Integer(day), month, new Integer(year)}
    }

    public static void flexiPrint(Object ... data) { // Object[]
        out.println("No. of elements: " + data.length);
        for (int i = 0; i < data.length; i++) {
            out.print(data[i] + " ");
        }
        out.println();
    }
}

```

The output:

```

No. of elements: 0

No. of elements: 1
1

No. of elements: 2
1 February

No. of elements: 3
1 February 2005

```

Overloading resolution

Resolution of overloaded methods selects the most specific method for execution.

One method is more specific than another method if all actual parameters that can be accepted by the one can be accepted by the other. A method call can lead to an ambiguity between two or more overloaded methods, and is flagged by the compiler. Example:

```

...
// The method 'flipFlop(String, int, Integer)' is ambiguous for the type VarargsDemo
flipFlop("(String, Integer, int)", new Integer(4), 2004); // COMPILER ERROR!
...

private static void flipFlop(String str, int i, Integer iRef) {
    out.println(str + " ==> (String, int, Integer)");
}

private static void flipFlop(String str, int i, int j) {
    out.println(str + " ==> (String, int, int)");
}

```

This is a legal example:

```

...
flipFlop("(String, Integer, int)", new Integer(4), 2004); // OK
...

private static void flipFlop(String str, Integer iRef, int i) {
    out.println(str + " ==> (String, Integer, int)");
}

```

```
private static void flipFlop(String str, int i, int j) {
    out.println(str + " ==> (String, int, int)");
}
```

The output will be:

```
(String, Integer, int) ==> (String, Integer, int)
```

Varargs and overloading

The example illustrates how the most specific overloaded method is chosen for a method call:

```
public class VarargsOverloading {

    public void operation(String str) {
        String signature = "(String)";
        out.println(str + " => " + signature);
    }

    public void operation(String str, int m) {
        String signature = "(String, int)";
        out.println(str + " => " + signature);
    }

    public void operation(String str, int m, int n) {
        String signature = "(String, int, int)";
        out.println(str + " => " + signature);
    }

    public void operation(String str, Integer... data) {
        String signature = "(String, Integer[])";
        out.println(str + " => " + signature);
    }

    public void operation(String str, Number... data) {
        String signature = "(String, Number[])";
        out.println(str + " => " + signature);
    }

    public void operation(String str, Object... data) {
        String signature = "(String, Object[])";
        out.println(str + " => " + signature);
    }

    public static void main(String[] args) {
        VarargsOverloading ref = new VarargsOverloading();
        ref.operation("1. (String)");
        ref.operation("2. (String, int)", 10);
        ref.operation("3. (String, Integer)", new Integer(10));
        ref.operation("4. (String, int, byte)", 10, (byte) 20);
        ref.operation("5. (String, int, int)", 10, 20);
        ref.operation("6. (String, int, long)", 10, 20L);
        ref.operation("7. (String, int, int, int)", 10, 20, 30);
        ref.operation("8. (String, int, double)", 10, 20.0);
        ref.operation("9. (String, int, String)", 10, "what?");
        ref.operation("10.(String, boolean)", false);
    }
}
```

The output:

```
1. (String) => (String)
2. (String, int) => (String, int)
3. (String, Integer) => (String, int)
4. (String, int, byte) => (String, int, int)
5. (String, int, int) => (String, int, int)
6. (String, int, long) => (String, Number[])
7. (String, int, int, int) => (String, Integer[])
8. (String, int, double) => (String, Number[])
9. (String, int, String) => (String, Object[])
10.(String, boolean) => (String, Object[])
```

Varargs and overriding

Overriding of varargs methods does not present any surprises as long as criteria for overriding is satisfied.

```
public class OneSuperclass {
    public int doIt(String str, Integer... data) throws java.io.EOFException,
        java.io.FileNotFoundException { // (1)
        String signature = "(String, Integer[])";
        out.println(str + " => " + signature);
        return 1;
    }

    public void doIt(String str, Number... data) { // (2)
        String signature = "(String, Number[])";
        out.println(str + " => " + signature);
    }
}
```

```
import static java.lang.System.out;

public class OneSubclass extends OneSuperclass {

    // public int doIt(String str, Integer[] data) // Overridden (a)
    public int doIt(String str, Integer... data) // Overridden (b)
        throws java.io.FileNotFoundException {
        String signature = "(String, Integer[])";
        out.println("Overridden: " + str + " => " + signature);
        return 0;
    }

    public void doIt(String str, Object... data) { // Overloading
        String signature = "(String, Object[])";
        out.println(str + " => " + signature);
    }

    public static void main(String[] args) throws Exception {
        OneSubclass ref = new OneSubclass();
        ref.doIt("1. (String)");
        ref.doIt("2. (String, int)", 10);
        ref.doIt("3. (String, Integer)", new Integer(10));
        ref.doIt("4. (String, int, byte)", 10, (byte) 20);
        ref.doIt("5. (String, int, int)", 10, 20);
        ref.doIt("6. (String, int, long)", 10, 20L);
        ref.doIt("7. (String, int, int, int)", 10, 20, 30);
        ref.doIt("8. (String, int, double)", 10, 20.0);
        ref.doIt("9. (String, int, String)", 10, "what?");
        ref.doIt("10.(String, boolean)", false);
    }
}
```

```
}
```

The output:

```
Overridden: 1. (String) => (String, Integer[])
Overridden: 2. (String, int) => (String, Integer[])
Overridden: 3. (String, Integer) => (String, Integer[])
4. (String, int, byte) => (String, Number[])
Overridden: 5. (String, int, int) => (String, Integer[])
6. (String, int, long) => (String, Number[])
Overridden: 7. (String, int, int, int) => (String, Integer[])
8. (String, int, double) => (String, Number[])
9. (String, int, String) => (String, Object[])
10.(String, boolean) => (String, Object[])
```

Another method signature:

```
...
public int doIt(String str, Integer[] data) // Overridden (a)
// public int doIt(String str, Integer... data) // Overridden (b)
...
```

However, the method will be invoked from superclass:

```
1. (String) => (String, Number[])
2. (String, int) => (String, Number[])
3. (String, Integer) => (String, Number[])
4. (String, int, byte) => (String, Number[])
5. (String, int, int) => (String, Number[])
6. (String, int, long) => (String, Number[])
7. (String, int, int, int) => (String, Number[])
8. (String, int, double) => (String, Number[])
9. (String, int, String) => (String, Object[])
10.(String, boolean) => (String, Object[])
```

[Prev](#)

Develop code that declares, initializes, and uses primitives, arrays, enums, and objects as static, instance, and local variables. Also, use legal identifiers for variable names.

[Up](#)

[Home](#)

[Next](#)

Given a code example, determine if a method is correctly overriding or overloading another method, and identify legal return values (including covariant returns), for the method.



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

IBM Test 287
Study Guide